Date :- 17/09/2025

# Task 8.1

Implement python generator and decorators

a. Fibonacci seaquence Gererator

AIM: To write a python program that generates Fibonacci numbers up to a given limit using a generator function and displayy them to the User.

## ALGORITHM:

1. Start
2. Define a generator function
   fibonacci(n):
   - Initialize two variables a = 0, b = 1.
   - Loop n times:
     - ⇒ Yield the value of a.
     - ⇒ Update a and b = b, a+b.
3. Ask the user to input the number of terms n.
4. Use the generator function to generate Fibonacci numbers up to n.
5. Print the seaquence.
6. End.

OUTPUT

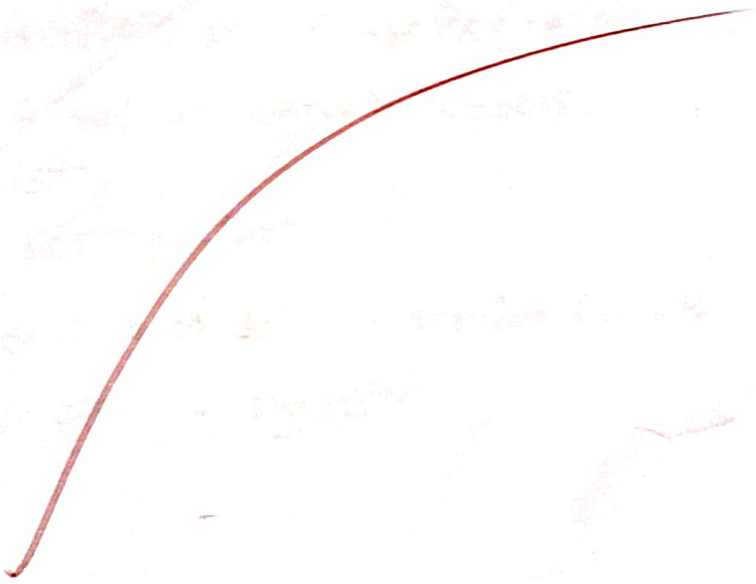
enter how many numbers you want: 7

Fibonacci sequence:

0 1 1 2 3 5 8

Fibonacci sequence iterators using generators

```
def fibonacci_generator(n):
    """A generator function to yield Fibonacci numbers
    upto n terms"""
    a, b = 0, 1
    count = 0
    while count < n:
        yield a
        a, b = b, a+b
        count += 1
num = int(input("enter how many numbers you want: "))
print("Fibonacci sequence:")
for value in fibonacci_sequence_generator(num):
    print(value, end = " ")
```



RESULT- Thus, the program successfully creates a generator function that produces fibonacci numbers upto the specified

Task 8-2

b. Function Execution Time Decorator

AIM:-

To write a python program that implements a decorator to calculate the execution time of any function and apply it to a sorting function.

ALGORITHM:-

1- Start

2. Import time module for measuring execution time

3. Define a decorator execution_time(func):
   - Define a wrapper function
   - Record start time.
   - call the original function.
   - Record end time.
   - print the execution time.

4- Define a function sort_numbers():
   - generate a list of random numbers.
   - sort the list
   - print the sorted list

5. Apply the decorator to the sorting function.

6- call the decorated function.

7. End

Sample output:

Unsorted list: [583, 12, 947, 76, 321, 65, 876, 102, 45, 703,
210, 999, 34, 88, 654, 400, 72, 500, 201, 305]

Sorted list: [12, 34, 45, 65, 72, 76, 88, 102, 201, 210, 305,
321, 400, 500, 583, 654, 703, 876, 947, 999]

Execution time : 0.000021 seconds.

Program.

```
import time
import random

# Decorator to measure execution time
def execution_time(func):
    def wrapper(*args, **kwargs):
        start = time.time()
        result = func(*args, **kwargs)
        end = time.time()
        print(f"\n Execution time: {end-start:.6f} seconds")
        return result
    return wrapper

# Sorting function with decorator
@execution_time
def sort_numbers():
    numbers = [random.randint(1, 1000)
for _ in range(20)]
    print("Unsorted list:", numbers)
    numbers.sort()
    print("Sorted list:", numbers)
```

| VEL TECH - CSE | |
|---|---|
| EX. NO. | 8 |
| PERFORMANCE (5) | 5 |
| RESULT AND ANALYSIS (5) | 5 |
| VIVA VOCE (5) | 5 |
| RECORD (5) | 5 |
| TOTAL (20) | 25 |
| SIGNATURE | |

RESULT: Thus, the decorator python program to implement python decorators was successfully executed and output was verified.